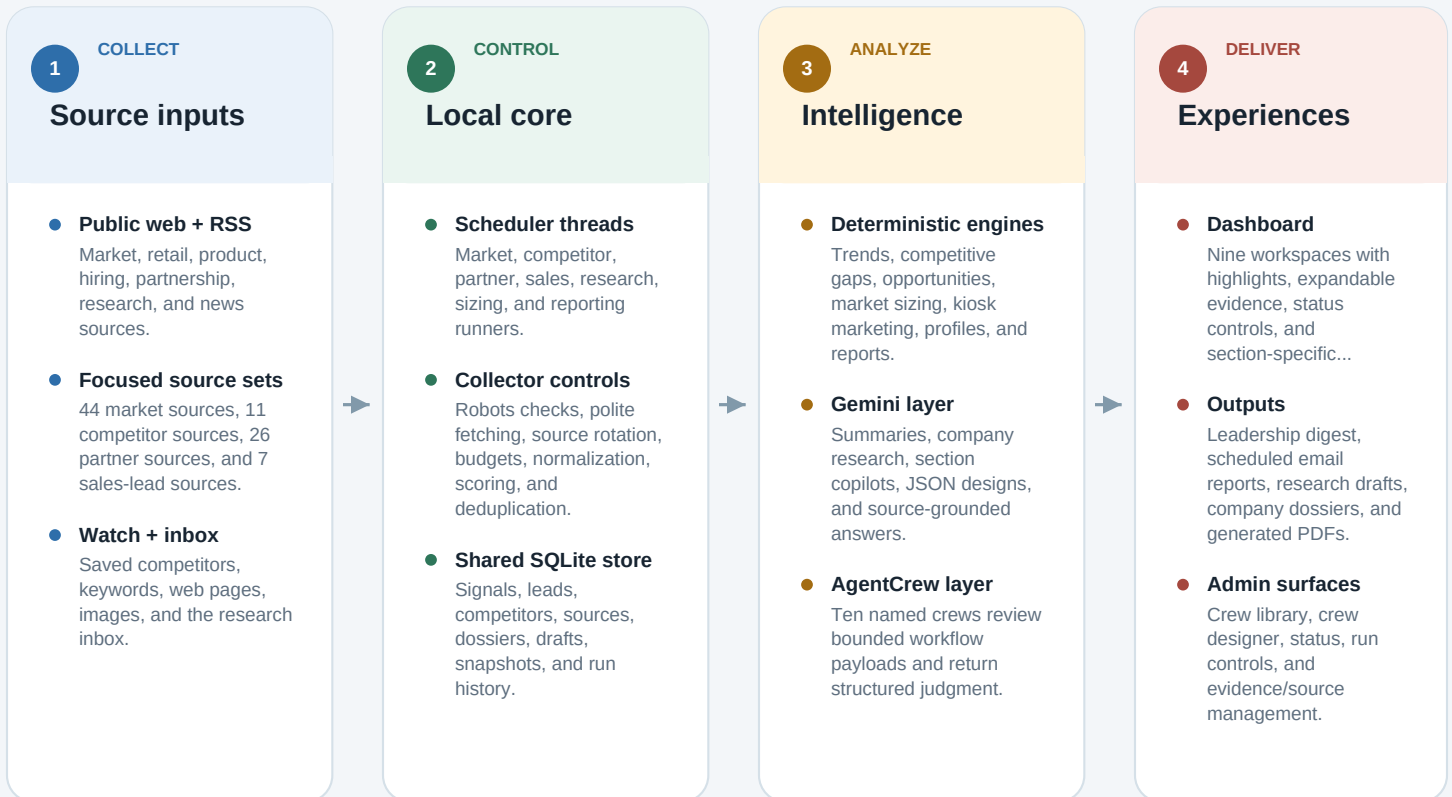


How the intelligence system is connected

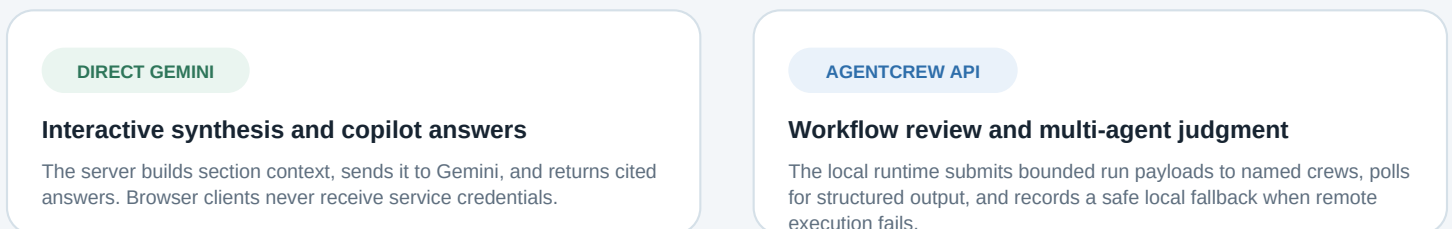
A local-first pipeline connects configured public sources, scheduled collectors, a shared SQLite knowledge base, deterministic analysis, two AI layers, and the browser dashboard.

Design principle: evidence and workflow state stay local

The local Python application owns source collection, scoring, storage, scheduling, analysis, and delivery. External AI receives bounded context for synthesis or review; it does not own the database or the run schedule.



Two distinct AI connections

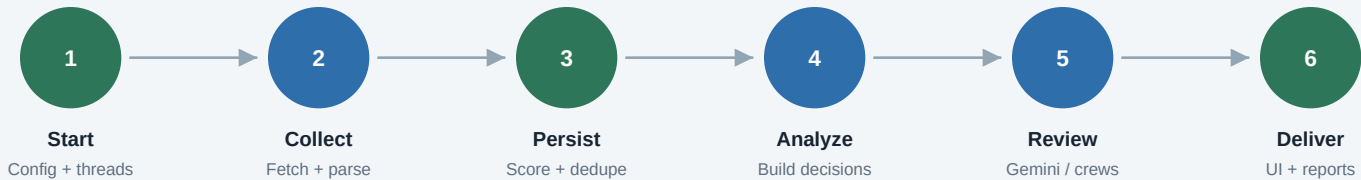


Primary implementation: cli.py, scheduler.py, storage.py, dashboard.py, llm.py, agentcrew.py, multi_agent_jobs.py

What runs, when, and where data moves

Each scheduled runner follows the same control loop: collect within a budget, write normalized records locally, compute analysis, optionally request crew review, then expose the result through APIs and the dashboard.

The local run loop



Configured background schedules

360m	Market intelligence Three collection passes, SQLite write, digest, then market crew review.	72m	Competitor intelligence Rotating feed sources, watchlist/browser/profile work, then crew review.
90m	Partners + resellers Rotating partner sources, contact evidence, qualification, and crew review.	120m	Sales leads Business triggers, product-fit candidates, evidence profiles, and crew review.
60m	Research automation Saved searches, collections, drafts, dossiers, inbox, and crew review.	Wkly	Sizing + reporting Market-sizing evidence reviews and scheduled leadership email preparation.

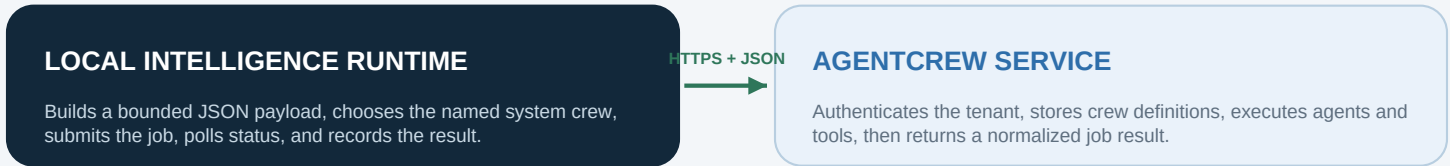
290 stored signals	14 signals added today	10/10 crews registered	Runtime Snapshot - 04 Aug 2026 No remote execution verified
------------------------------	----------------------------------	----------------------------------	---

Operational truth

FRESHNESS + AVAILABILITY - Schedulers run only while the Python server remains open and the host machine is awake. - The UI polls runner status every 30 seconds; executive metrics refresh on page load or when the user clicks Refresh. - Robots rules, 404 pages, source limits, and missing browser tooling can reduce coverage without stopping the system.	SECURITY + RESILIENCY - API credentials stay server-side and are loaded from environment settings; the browser calls local endpoints only. - SQLite remains the source of record. AgentCrew is configured with database_writeback=false. - If remote crew execution fails, collection, persistence, analysis, and delivery complete through the local safeguard path.
---	--

How the crew layer is connected through the API

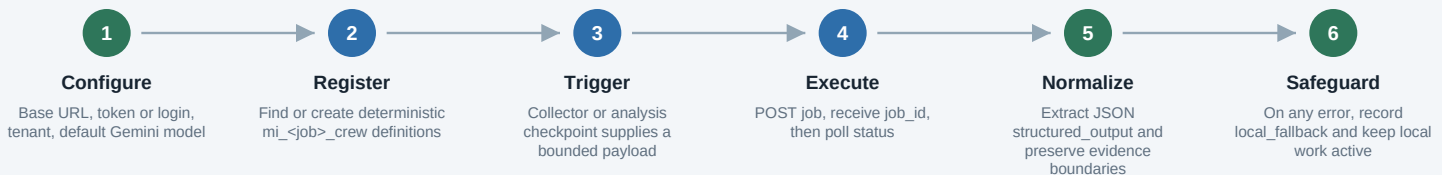
AgentCrew is an external review service attached to local workflow checkpoints. It adds role-based judgment and structured review, while the local application keeps deterministic control of data, schedules, and delivery.



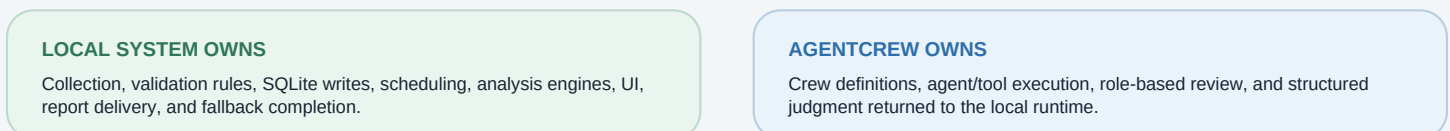
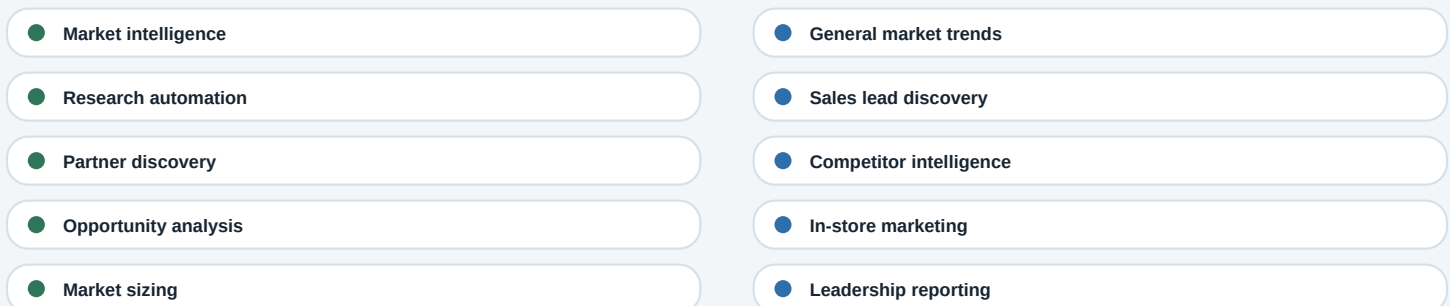
API contract used by the integration

METHOD	REMOTE ROUTE	HOW THE LOCAL CLIENT USES IT
POST	/api/v1/login	Username/password login; cache returned Bearer token.
GET	/api/v1/crew/tools	Discover tools available to crew definitions.
GET	/api/v1/crew?tenant=...	Resolve existing crews and stable crew IDs by tenant.
PUT	/api/v1/crew	Create a missing system crew from the Gemini-designed definition.
POST	/api/v1/crews	Submit crew_id, query, tenant, mode, token and temperature limits.
PATCH	/api/v1/crews?job_id=...	Poll every 2 seconds until complete, failed, or 180-second timeout.

Execution sequence



Ten system crews mapped to workflow checkpoints



CURRENT STATE 10/10 crews registered; remote execution is not yet verified. Recent remote jobs returned Not Found, so local_safeguard completed the workflow.